B-trees solutions

18-E01 Node capacity

Derivation and check

Nonroot keys range t−1=3 through2t−1=7. At7 keys,an internal node has at most8 child pointers. A leaf with6 keys is legal. Except for the empty tree,a root needs at least1 key,but may have fewer than a nonroot.

18-E02 Root split

Derivation and check

Median20 rises to new root,left[10],right[30].25>20 chooses right,and25<30 yields[25,30]. Final child sizes1 and2 lie within1..3. Only immediately after the split do both children have exactly1 key.

18-E03 I/O height

Derivation and check

Height counts edges. Maximum capacity is $(2t)^{h+1}-1$. Height3 permits200^4−1=1,599,999,999 keys; height2 permits only7,999,999. Minimum legal count is $2t^h-1$; height5 needs more than10^9,while height4 is possible. Thus h=3 or4,and a leaf search visits4 or5 pages,one fewer actual read with resident root. Occupancy determines the actual path. These are capacity/I/O bounds,not measured disk latency.

18-E04 Borrowing must retain subtrees

Derivation and check

The left child has t−1=1 key,the right2 and can lend. Parent20 moves down to the left end-position,sibling minimum30 rises to root,and right sibling becomes[40]. Its first subtree[25] must move to left[10,20] as the final child,giving leaves[5],[15],[25]. Before entering[5],merge[5]+parent10+[15] to[5,10,15],then delete5. Final root[30];left internal[20] with leaves[10,15],[25];right internal[40] with[35],[45]. Inorder10,15,20,25,30,35,40,45 loses only5;all leaves depth2. Leaving25 right of root30 violates global order.

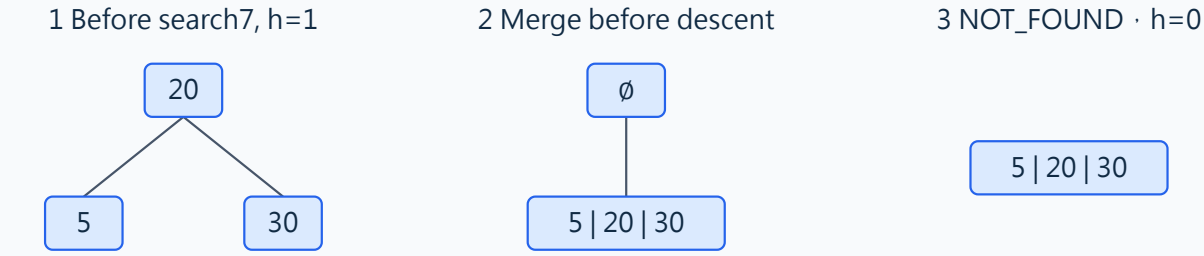
B-trees solutions (continued)

18-E05 Absent deletion may change shape

Derivation and check

Search7 chooses[5],but both it and sibling[30] have minimum size1. Merge with separator20 into leaf[5,20,30]. The old root has zero keys and one child. The merged leaf still lacks7,so return NOT_FOUND and replace the empty internal root by its sole child.

Failed deletion can still contract the root



7 is absent: merge both minimum leaves with20, then promote the empty root' s sole child.

Key/data mapping retained; inorder5,20,30 throughout. Edge height1→0.

The result is leaf root[5,20,30],height1→0. The whole key set and associated data remain unchanged. Failed deletion promises no removed key,not necessarily identical structural bits. An API requiring no writes on failure may search first,an extra contract with another path.

18-E06 Separate page comparisons from I/O

Derivation and check

Worst-case leaf search follows h edges and reads h nonroot pages with either method. Linear scanning needs at most2t−1 comparisons per page,CPU O((h+1)t);binary search uses O((h+1)log t). Binary search finds a position faster,but inserting in array nodes can still shiftΘ(t) keys/pointers,and splitting movesΘ(t) items. O(h) possible splits retain CPU O(t(h+1)) and I/O O(h+1). Different hardware/layouts require an explicit new model;do not confuse comparison and page-access denominators.

B-trees solutions (continued)

Verification code appendix: btree_checks.py

```
"""B-tree operations with invariant and sorted-set comparisons after every step."""
from bisect import bisect_left
from dataclasses import dataclass, field
from random import Random


@dataclass
class Node:
    keys: list = field(default_factory=list)
    children: list = field(default_factory=list)


class BTree:
    def __init__(self, degree=2):
        assert degree>=2
        self.t=degree
        self.root=Node()
        self.events=set()

    def search(self,key):
        node=self.root
        while True:
            i=bisect_left(node.keys,key)
            if i<len(node.keys) and node.keys[i]==key:
                return True
            if not node.children:
                return False
            node=node.children[i]

    def split(self,parent,i):
        t=self.t
        left=parent.children[i]
        median=left.keys[t-1]
        right=Node(left.keys[t:],left.children[t:] if left.children else [])
        left.keys=left.keys[:t-1]
        if left.children:
            left.children=left.children[:t]
        parent.keys.insert(i,median)
        parent.children.insert(i+1,right)
        self.events.add("split")

    def insert(self,key):
        if self.search(key):
            return False
        if len(self.root.keys)==2*self.t-1:
            self.root=Node([], [self.root])
            self.split(self.root,0)
            self.events.add("root-grow")
        node=self.root
        while node.children:
            i=bisect_left(node.keys,key)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

B-trees solutions (continued)

```
            if len(node.children[i].keys)==2*self.t-1:
                self.split(node,i)
                if key>node.keys[i]:
                    i+=1
            node=node.children[i]
        node.keys.insert(bisect_left(node.keys,key),key)
        return True

    def merge(self,parent,i):
        left,right=parent.children[i:i+2]
        left.keys.append(parent.keys.pop(i))
        left.keys.extend(right.keys)
        left.children.extend(right.children)
        parent.children.pop(i+1)
        self.events.add("merge")
        return left

    def delete(self,key):
        found=self._delete(self.root,key)
        if not self.root.keys and self.root.children:
            self.root=self.root.children[0]
            self.events.add("root-shrink")
        return found

    def _delete(self,node,key):
        i=bisect_left(node.keys,key)
        if i<len(node.keys) and node.keys[i]==key:
            if not node.children:
                node.keys.pop(i)
                self.events.add("leaf-delete")
                return True
            left,right=node.children[i:i+2]
            if len(left.keys)>=self.t:
                cursor=left
                while cursor.children:
                    cursor=cursor.children[-1]
                predecessor=cursor.keys[-1]
                self._delete(left,predecessor)
                node.keys[i]=predecessor
                self.events.add("predecessor")
            elif len(right.keys)>=self.t:
                cursor=right
                while cursor.children:
                    cursor=cursor.children[0]
                successor=cursor.keys[0]
                self._delete(right,successor)
                node.keys[i]=successor
                self.events.add("successor")
            else:
                self._delete(self.merge(node,i),key)
                self.events.add("internal-merge")
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

B-trees solutions (continued)

```
            return True
        if not node.children:
            return False
        child=node.children[i]
        if len(child.keys)==self.t-1:
            if i>0 and len(node.children[i-1].keys)>=self.t:
                sibling=node.children[i-1]
                child.keys.insert(0,node.keys[i-1])
                node.keys[i-1]=sibling.keys.pop()
                if sibling.children:
                    child.children.insert(0,sibling.children.pop())
                self.events.add("borrow-left")
            elif i<len(node.keys) and len(node.children[i+1].keys)>=self.t:
                sibling=node.children[i+1]
                child.keys.append(node.keys[i])
                node.keys[i]=sibling.keys.pop(0)
                if sibling.children:
                    child.children.append(sibling.children.pop(0))
                self.events.add("borrow-right")
            elif i<len(node.keys):
                child=self.merge(node,i)
            else:
                child=self.merge(node,i-1)
        return self._delete(child,key)

    def validate(self):
        leaf_depths=[]
        seen=set()
        def visit(node,low,high,depth,root=False):
            assert id(node) not in seen
            seen.add(id(node))
            assert node.keys==sorted(set(node.keys))
            assert all(low<key<high for key in node.keys)
            assert len(node.keys)<=2*self.t-1
            assert root or len(node.keys)>=self.t-1
            if not node.children:
                leaf_depths.append(depth)
                return node.keys[:]
            assert len(node.children)==len(node.keys)+1
            assert node.keys or not root
            bounds=[low]+node.keys+[high]
            result=[]
            for i,child in enumerate(node.children):
                result.extend(visit(child,bounds[i],bounds[i+1],depth+1))
                if i<len(node.keys):
                    result.append(node.keys[i])
            return result
        result=visit(self.root,float("-inf"),float("inf"),0,True)
        assert len(set(leaf_depths))==1
        return result
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

B-trees solutions (continued)

```
def main():
    tree=BTree()
    for key in (10,20,30,25):
        tree.insert(key)
        tree.validate()
    assert tree.root.keys==[20]
    assert [node.keys for node in tree.root.children]==[[10],[25,30]]
    borrow=BTree()
    borrow.root=Node([20],[Node([10],[Node([5]),Node([15])]),Node([30,40],
[Node([25]),Node([35]),Node([45])])])
    assert borrow.validate()==[5,10,15,20,25,30,35,40,45]
    assert borrow.delete(5)
    assert borrow.validate()==[10,15,20,25,30,35,40,45]
    assert borrow.root.keys==[30]
    assert borrow.root.children[0].keys==[20]

    events=set()
    operations=0
    for degree in (2,3,5):
        for seed in range(8):
            rng=Random(seed)
            tree=BTree(degree)
            expected=set()
            for _ in range(350):
                key=rng.randrange(120)
                if rng.random()<0.55:
                    assert tree.insert(key)==(key not in expected)
                    expected.add(key)
                else:
                    assert tree.delete(key)==(key in expected)
                    expected.discard(key)
                assert tree.validate()==sorted(expected)
                operations+=1
            for key in list(expected):
                assert tree.delete(key)
                expected.remove(key)
                assert tree.validate()==sorted(expected)
                operations+=1
            events.update(tree.events)
    required={"split","root-grow","leaf-delete","merge","root-
shrink","predecessor","successor","internal-merge","borrow-left","borrow-right"}
    assert required<=events,(required-events)
    assert 200**3-1<10**9<=200**4-1
    assert 2*100**4-1<=10**9<2*100**5-1
    print(f"PASS BTree_mixed_and_drain_operations={operations} branches={sorted(events)}
height_n1e9_t100=3_or4")


if __name__=="__main__":
    main()
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

B-trees solutions (continued)

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study